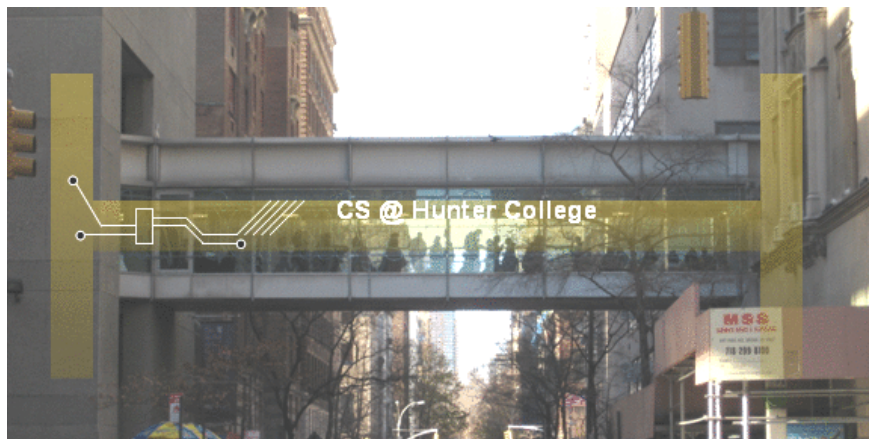


CSci 127: Introduction to Computer Science



hunter.cuny.edu/csci

Today's Topics



- Recap: Plotly
- Indefinite loops
- Design Patterns: Max (Min)
- Design Challenge
- Dictionaries

Today's Topics



- **Recap: Plotly**
- Indefinite loops
- Design Patterns: Max (Min)
- Design Challenge
- Dictionaries

Plotly.py



- An open-source graphing library for Python.

Plotly.py



- An open-source graphing library for Python.
- Built on top of the JavaScript `plotly.js`.

Plotly.py



- An open-source graphing library for Python.
- Built on top of the JavaScript `plotly.js`.
- Can build stand-alone & browser-based charts, graphs, and maps.
- We will use the simplified interface: Plotly Express (`px`).

In Pairs or Triples:

What does this code do?

```
import plotly.express as px #Import the plotly express package for making maps
import pandas as pd         #We're using Pandas to organize the data

data = {
    'city': ['New York', 'Los Angeles', 'Paris', 'Tokyo', 'Rio de Janeiro'],
    'latitude': [40.7128, 34.0549, 48.8575, 33.8688, -22.9068],
    'longitude': [-74.0060, -118.2426, 2.3514, 139.6500, -43.1729],
    'population': [8_258_000, 3_821_000, 2_103_000, 14_180_000, 6_211_000]
}
df = pd.DataFrame(data)

# Create the scatter plot on a map
fig = px.scatter_geo(df,
    lat="latitude",
    lon="longitude",
    size="population", # Size of the markers based on population
    hover_name="city", # Display city name on hover
    projection="natural earth", # Map projection
    title="City Populations")

fig.show() #Shows the map in a browser
fig.write_html('world_map_px.html') #
```

Plotly example

What does this code do?

```
import plotly.express as px #Import the plotly express package for making maps
import pandas as pd        #We're using Pandas to organize the data

data = {
    'city': ['New York', 'Los Angeles', 'Paris', 'Tokyo', 'Rio de Janeiro'],
    'latitude': [40.7128, 34.0549, 48.8575, 33.8688, -22.9068],
    'longitude': [-74.0060, -118.2426, 2.3514, 139.6500, -43.1729],
    'population': [8_258_000, 3_821_000, 2_103_000, 14_180_000, 6_211_000]
}
df = pd.DataFrame(data)

# Create the scatter plot on a map
fig = px.scatter_geo(df,
                    lat="latitude",
                    lon="longitude",
                    size="population", # Size of the markers based on population
                    hover_name="city", # Display city name on hover
                    projection="natural earth", # Map projection
                    title="City Populations")

fig.show() #Shows the map in a browser
fig.write_html('world_map_px.html') #
```


Plotly example

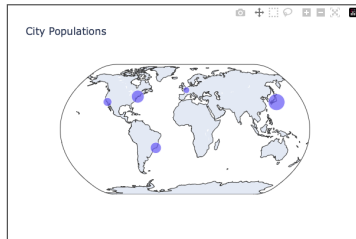
What does this code do?

```
import plotly.express as px #Import the plotly express package for making maps
import pandas as pd        #We're using Pandas to organize the data

data = {
    'city': ['New York', 'Los Angeles', 'Paris', 'Tokyo', 'Rio de Janeiro'],
    'latitude': [40.7128, 34.0549, 48.8575, 33.8688, -22.9068],
    'longitude': [-74.0060, -118.2426, 2.3514, 139.6500, -43.1729],
    'population': [8_258_000, 3_821_000, 2_103_000, 14_180_000, 6_211_000]
}
df = pd.DataFrame(data)

# Create the scatter plot on a map
fig = px.scatter_geo(df,
                    lat="latitude",
                    lon="longitude",
                    size="population", # Size of the markers based on population
                    hover_name="city", # Display city name on hover
                    projection="natural earth", # Map projection
                    title="City Populations")

fig.show() #Shows the map in a browser
fig.write_html('world_map_px.html') #
```



Today's Topics



- Recap: Plotly
- **Indefinite loops**
- Design Patterns: Max (Min)
- Design Challenge
- Dictionaries

Challenge:

- Write a function that asks a user for number after 2000 but before 2018. The function should repeatedly ask the user for a number until they enter one within the range and return the number.

Coding

- Write a function that asks a user for number after 2000 but before 2018. The function should repeatedly ask the user for a number until they enter one within the range and return the number..

Coding

- Write a function that asks a user for number after 2000 but before 2018. The function should repeatedly ask the user for a number until they enter one within the range and return the number.

```
def getYear():
```

Coding

- Write a function that asks a user for number after 2000 but before 2018. The function should repeatedly ask the user for a number until they enter one within the range and return the number.

```
def getYear():
```

```
    return(num)
```

Coding

- Write a function that asks a user for number after 2000 but before 2018. The function should repeatedly ask the user for a number until they enter one within the range and return the number.

```
def getYear():  
    num = 0  
  
    return(num)
```

Coding

- Write a function that asks a user for number after 2000 but before 2018. The function should repeatedly ask the user for a number until they enter one within the range and return the number.

```
def getYear():  
    num = 0  
    while num <= 2000 or num >= 2018:  
  
    return(num)
```


Coding

- Write a function that asks a user for number after 2000 but before 2018. The function should repeatedly ask the user for a number until they enter one within the range and return the number.

```
def getYear():  
    num = 0  
    while num <= 2000 or num >= 2018:  
        num = int(input('Enter a number > 2000 & < 2018'))  
  
    return(num)
```

Indefinite Loops

```
#Spring 2012 Final Exam, #8  
nums = [1,4,0,6,5,2,9,8,12]  
print(nums)  
i=0  
while i < len(nums)-1:  
    if nums[i] < nums[i+1]:  
        nums[i], nums[i+1] = nums[i+1], nums[i]  
        i=i+1  
print(nums)
```

Indefinite Loops

- Indefinite loops repeat as long as the condition is true.

```
#Spring 2012 Final Exam, #8  
nums = [1,4,0,6,5,2,9,8,12]  
print(nums)  
i=0  
while i < len(nums)-1:  
    if nums[i] < nums[i+1]:  
        nums[i], nums[i+1] = nums[i+1], nums[i]  
        i=i+1  
print(nums)
```

Indefinite Loops

- Indefinite loops repeat as long as the condition is true.
- Could execute the body of the loop zero times, 10 times, infinite number of times.

```
#Spring 2012 Final Exam, #8  
nums = [1,4,0,6,5,2,9,8,12]  
print(nums)  
i=0  
while i < len(nums)-1:  
    if nums[i] < nums[i+1]:  
        nums[i], nums[i+1] = nums[i+1], nums[i]  
        i=i+1  
print(nums)
```

Indefinite Loops

```
#Spring 2012 Final Exam, #8  
nums = [1,4,0,6,5,2,9,8,12]  
print(nums)  
i=0  
while i < len(nums)-1:  
    if nums[i] < nums[i+1]:  
        nums[i], nums[i+1] = nums[i+1], nums[i]  
        i=i+1  
print(nums)
```

- Indefinite loops repeat as long as the condition is true.
- Could execute the body of the loop zero times, 10 times, infinite number of times.
- The condition determines how many times.

Indefinite Loops

```
#Spring 2012 Final Exam, #8  
nums = [1,4,0,6,5,2,9,8,12]  
print(nums)  
i=0  
while i < len(nums)-1:  
    if nums[i] < nums[i+1]:  
        nums[i], nums[i+1] = nums[i+1], nums[i]  
        i=i+1  
print(nums)
```

- Indefinite loops repeat as long as the condition is true.
- Could execute the body of the loop zero times, 10 times, infinite number of times.
- The condition determines how many times.
- Very useful for checking input, simulations, and games.

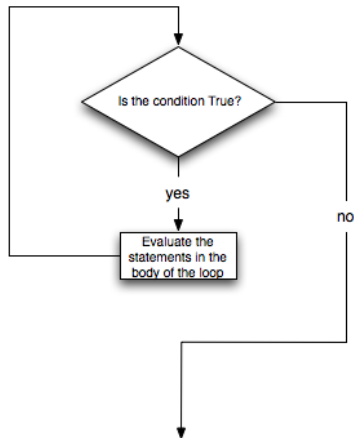
Indefinite Loops

```
#Spring 2012 Final Exam, #8  
nums = [1,4,0,6,5,2,9,8,12]  
print(nums)  
i=0  
while i < len(nums)-1:  
    if nums[i] < nums[i+1]:  
        nums[i], nums[i+1] = nums[i+1], nums[i]  
        i=i+1  
print(nums)
```

Indefinite Loops

#Spring 2012 Final Exam, #8

```
nums = [1,4,0,6,5,2,9,8,12]
print(nums)
i=0
while i < len(nums)-1:
    if nums[i] < nums[i+1]:
        nums[i], nums[i+1] = nums[i+1], nums[i]
    i=i+1
print(nums)
```



Challenge

Predict what this code does:

```
#Random search
import turtle
import random
tess = turtle.Turtle()
tess.color('steelBlue')
tess.shape('turtle')
tess.penup()
#Start off screen:
tess.goto(-250,-250)
#Remember: abs(x) < 25 means absolute value: -25 < x < 25
while abs(tess.xcor()) > 25 or abs(tess.ycor()) > 25:
    x = random.randrange(-200,200)
    y = random.randrange(-200,200)
    tess.goto(x,y)
    tess.stamp()
    print(tess.xcor(), tess.ycor())
print('Found the center!')
```

Trinket Demo

```
#Random search
import turtle
import random
tess = turtle.Turtle()
tess.color('steelBlue')
tess.shape('turtle')
tess.penup()
#Start off screen:
tess.goto(-250,-250)
#Remember: abs(x) < 25 means absolute value: -25 < x < 25
while abs(tess.xcor()) > 25 or abs(tess.ycor()) > 25:
    x = random.randrange(-200,200)
    y = random.randrange(-200,200)
    tess.goto(x,y)
    tess.stamp()
    print(tess.xcor(), tess.ycor())
print('Found the center!')
```

(Demo with trinket)

Today's Topics



- Recap: Plotly
- Indefinite loops
- **Design Patterns: Max (Min)**
- Design Challenge
- Dictionaries

Design Patterns

- A **design pattern** is a standard algorithm or approach for solving a common problem.



Design Patterns



- A **design pattern** is a standard algorithm or approach for solving a common problem.
- The pattern is independent of the programming language.

Design Patterns



- A **design pattern** is a standard algorithm or approach for solving a common problem.
- The pattern is independent of the programming language.
- Can think of as a master recipe, with variations for different situations.

Design Question:



You can uncover one card at a time.
How would you go about finding the highest card?

Challenge:

Predict what the code will do:

```
nums = [1,4,10,6,5,42,9,8,12]

maxNum = 0
for n in nums:
    if n > maxNum:
        maxNum = n
print('The max is', maxNum)
```


Python Tutor

```
nums = [1,4,10,6,5,42,9,8,12]

maxNum = 0
for n in nums:
    if n > maxNum:
        maxNum = n
print('The max is', maxNum)
```

(Demo with pythonTutor)

Max Design Pattern

- Set a variable to the smallest value.

```
nums = [1,4,10,6,5,42,9,8,12]

maxNum = 0
for n in nums:
    if n > maxNum:
        maxNum = n
print('The max is', maxNum)
```

Max Design Pattern

- Set a variable to the smallest value.
- Loop through the list,

```
nums = [1,4,10,6,5,42,9,8,12]

maxNum = 0
for n in nums:
    if n > maxNum:
        maxNum = n
print('The max is', maxNum)
```

Max Design Pattern

- Set a variable to the smallest value.
- Loop through the list,
- If the current number is larger, update your variable.

```
nums = [1,4,10,6,5,42,9,8,12]

maxNum = 0
for n in nums:
    if n > maxNum:
        maxNum = n
print('The max is', maxNum)
```

Max Design Pattern

```
nums = [1,4,10,6,5,42,9,8,12]
maxNum = 0
for n in nums:
    if n > maxNum:
        maxNum = n
print('The max is', maxNum)
```

- Set a variable to the smallest value.
- Loop through the list,
 - If the current number is larger, update your variable.
- Print/return the largest number found.

Max Design Pattern

```
nums = [1,4,10,6,5,42,9,8,12]

maxNum = 0
for n in nums:
    if n > maxNum:
        maxNum = n
print('The max is', maxNum)
```

- Set a variable to the smallest value.
- Loop through the list,
 - If the current number is larger, update your variable.
- Print/return the largest number found.
- Must look at entire list to determine max is found

Max Design Pattern

```
nums = [1,4,10,6,5,42,9,8,12]

maxNum = 0
for n in nums:
    if n > maxNum:
        maxNum = n
print('The max is', maxNum)
```

- Set a variable to the smallest value.
- Loop through the list,
 - If the current number is larger, update your variable.
- Print/return the largest number found.
- Must look at entire list to determine max is found
- Similar idea works for finding the minimum value.

Max Design Pattern

```
nums = [1,4,10,6,5,42,9,8,12]

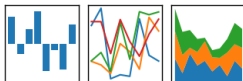
maxNum = 0
for n in nums:
    if n > maxNum:
        maxNum = n
print('The max is', maxNum)
```

- Set a variable to the smallest value.
- Loop through the list,
 - If the current number is larger, update your variable.
- Print/return the largest number found.
- Must look at entire list to determine max is found
- Similar idea works for finding the minimum value.
- Different from **Linear Search**: can stop when value you are looking for is found.

Pandas: Minimum Values

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$

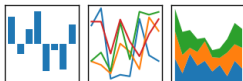


- In Pandas, lovely built-in functions:

Pandas: Minimum Values

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$

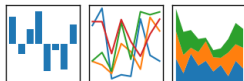


- In Pandas, lovely built-in functions:
 - ▶ `df.sort_values('First Name')` and
 - ▶ `df['First Name'].min()`

Pandas: Minimum Values

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$



- In Pandas, lovely built-in functions:
 - ▶ `df.sort_values('First Name')` and
 - ▶ `df['First Name'].min()`
- What if you don't have a CSV and DataFrame, or data not ordered?

Design Question: Find first alphabetically

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$

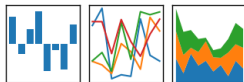


- What if you don't have a CSV and DataFrame, or data not ordered?

Design Question: Find first alphabetically

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$

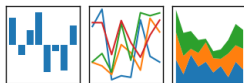


- What if you don't have a CSV and DataFrame, or data not ordered?
- Useful *Design Pattern*: min/max

Design Question: Find first alphabetically

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$

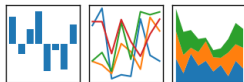


- What if you don't have a CSV and DataFrame, or data not ordered?
- Useful *Design Pattern*: min/max
 - ▶ Set a variable to worst value (i.e. `maxN = 0` or `first = "ZZ"`).

Design Question: Find first alphabetically

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$

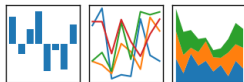


- What if you don't have a CSV and DataFrame, or data not ordered?
- Useful *Design Pattern*: min/max
 - ▶ Set a variable to worst value (i.e. `maxN = 0` or `first = "ZZ"`).
 - ▶ For each item, X, in the list:

Design Question: Find first alphabetically

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$

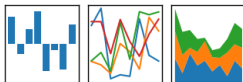


- What if you don't have a CSV and DataFrame, or data not ordered?
- Useful *Design Pattern*: min/max
 - ▶ Set a variable to worst value (i.e. `maxN = 0` or `first = "ZZ"`).
 - ▶ For each item, X, in the list:
 - ★ Compare X to your variable.

Design Question: Find first alphabetically

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$

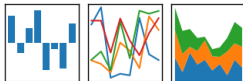


- What if you don't have a CSV and DataFrame, or data not ordered?
- Useful *Design Pattern*: min/max
 - ▶ Set a variable to worst value (i.e. `maxN = 0` or `first = "ZZ"`).
 - ▶ For each item, X, in the list:
 - ★ Compare X to your variable.
 - ★ If better, update your variable to be X.

Design Question: Find first alphabetically

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$



- What if you don't have a CSV and DataFrame, or data not ordered?
- Useful *Design Pattern*: min/max
 - ▶ Set a variable to worst value (i.e. `maxN = 0` or `first = "ZZ"`).
 - ▶ For each item, X, in the list:
 - ★ Compare X to your variable.
 - ★ If better, update your variable to be X.
 - ▶ Print/return X.

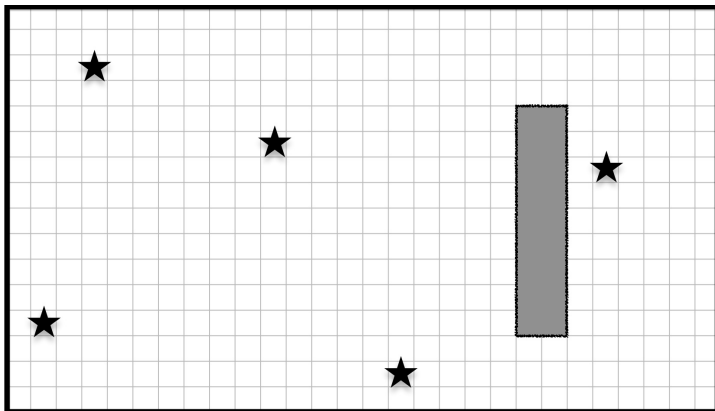
Today's Topics



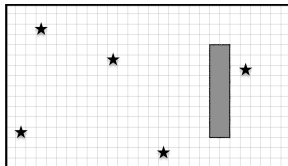
- Recap: Plotly
- Indefinite loops
- Design Patterns: Max (Min)
- **Design Challenge**
- Dictionaries

Design Challenge

Collect all five stars (locations randomly generated):

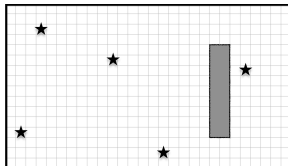


Design Challenge



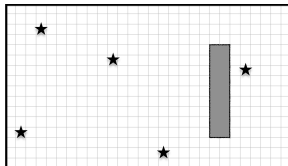
- Possible approaches:

Design Challenge



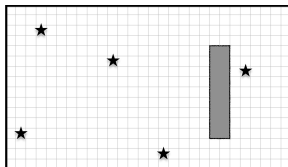
- Possible approaches:
 - ▶ Randomly wander until all 5 collected, or

Design Challenge



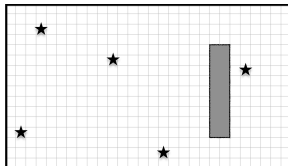
- Possible approaches:
 - ▶ Randomly wander until all 5 collected, or
 - ▶ Start in one corner, and systematically visit every point until 5 stars found (**Linear Search**).

Design Challenge



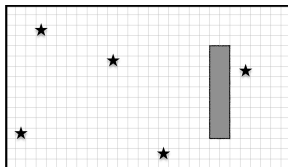
- Possible approaches:
 - ▶ Randomly wander until all 5 collected, or
 - ▶ Start in one corner, and systematically visit every point until 5 stars found (**Linear Search**).
- **Input:** The map of the 'world.'

Design Challenge



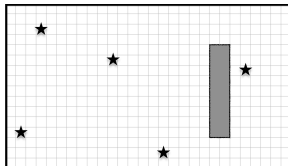
- Possible approaches:
 - ▶ Randomly wander until all 5 collected, or
 - ▶ Start in one corner, and systematically visit every point until 5 stars found (**Linear Search**).
- **Input:** The map of the 'world.'
- **Output:** Time taken and/or locations of the 5 stars.

Design Challenge



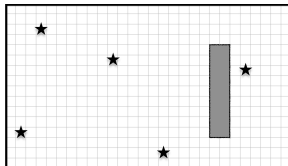
- Possible approaches:
 - ▶ Randomly wander until all 5 collected, or
 - ▶ Start in one corner, and systematically visit every point until 5 stars found (**Linear Search**).
- **Input:** The map of the 'world.'
- **Output:** Time taken and/or locations of the 5 stars.
- How to store locations? Use `numpy` array with -1 everywhere.

Design Challenge



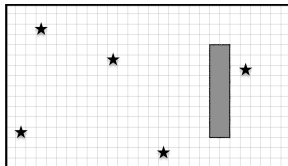
- Possible approaches:
 - ▶ Randomly wander until all 5 collected, or
 - ▶ Start in one corner, and systematically visit every point until 5 stars found (**Linear Search**).
- **Input:** The map of the 'world.'
- **Output:** Time taken and/or locations of the 5 stars.
- How to store locations? Use `numpy` array with -1 everywhere.
- Possible algorithms: `while numStars < 5:`

Design Challenge



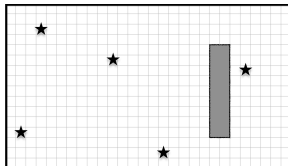
- Possible approaches:
 - ▶ Randomly wander until all 5 collected, or
 - ▶ Start in one corner, and systematically visit every point until 5 stars found (**Linear Search**).
- **Input:** The map of the 'world.'
- **Output:** Time taken and/or locations of the 5 stars.
- How to store locations? Use `numpy` array with -1 everywhere.
- Possible algorithms: while `numStars < 5`:
 - ▶ Move forward.

Design Challenge



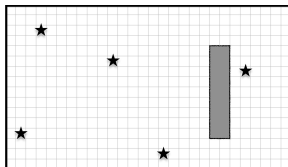
- Possible approaches:
 - ▶ Randomly wander until all 5 collected, or
 - ▶ Start in one corner, and systematically visit every point until 5 stars found (**Linear Search**).
- **Input:** The map of the 'world.'
- **Output:** Time taken and/or locations of the 5 stars.
- How to store locations? Use `numpy` array with -1 everywhere.
- Possible algorithms: `while numStars < 5:`
 - ▶ Move forward.
 - ▶ If wall, mark 0 in map, randomly turn left or right.

Design Challenge



- Possible approaches:
 - ▶ Randomly wander until all 5 collected, or
 - ▶ Start in one corner, and systematically visit every point until 5 stars found (**Linear Search**).
- **Input:** The map of the 'world.'
- **Output:** Time taken and/or locations of the 5 stars.
- How to store locations? Use `numpy` array with -1 everywhere.
- Possible algorithms: `while numStars < 5:`
 - ▶ Move forward.
 - ▶ If wall, mark 0 in map, randomly turn left or right.
 - ▶ If star, mark 1 in map and add 1 to numStars.

Design Challenge



- Possible approaches:
 - ▶ Randomly wander until all 5 collected, or
 - ▶ Start in one corner, and systematically visit every point until 5 stars found (**Linear Search**).
- **Input:** The map of the 'world.'
- **Output:** Time taken and/or locations of the 5 stars.
- How to store locations? Use `numpy` array with -1 everywhere.
- Possible algorithms: `while numStars < 5`:
 - ▶ Move forward.
 - ▶ If wall, mark 0 in map, randomly turn left or right.
 - ▶ If star, mark 1 in map and add 1 to numStars.
 - ▶ Otherwise, mark 2 in map that it's an empty square.

Today's Topics



- Recap: Plotly
- Indefinite loops
- Design Patterns: Max (Min)
- Design Challenge
- **Dictionaries**

Collection Types

More details: see [ThinkCS §9.2](#)



- Primitive (or atomic) Types:

Collection Types

More details: see [ThinkCS §9.2](#)



- **Primitive (or atomic) Types:**
 - ▶ can't be broken down into smaller pieces.

Collection Types

More details: see [ThinkCS §9.2](#)



- Primitive (or atomic) Types:
 - ▶ can't be broken down into smaller pieces.
 - ▶ Examples: `int`, `float`, `bool`
- Collection Types::

Collection Types

More details: see [ThinkCS §9.2](#)



- **Primitive (or atomic) Types:**
 - ▶ can't be broken down into smaller pieces.
 - ▶ Examples: `int`, `float`, `bool`
- **Collection Types:**
 - ▶ comprised of smaller pieces.

Collection Types

More details: see [ThinkCS §9.2](#)



- **Primitive (or atomic) Types:**
 - ▶ can't be broken down into smaller pieces.
 - ▶ Examples: `int`, `float`, `bool`
- **Collection Types:**
 - ▶ comprised of smaller pieces.
 - ▶ Examples: `list`, `strings`, but there are more...

Collection Types

More details: see [ThinkCS §9.2](#)

Collections are built up of smaller types:



Collection Types

More details: see [ThinkCS §9.2](#)

Collections are built up of smaller types:

- **lists**: ordered collection of 0 or more objects. Mutable.



Collection Types

More details: see [ThinkCS §9.2](#)

Collections are built up of smaller types:

- **lists**: ordered collection of 0 or more objects. Mutable. Ex: `lst = [1, "love", "Python!"]`
- **tuples**: ordered collection of 0 or more objects. Immutable.



Collection Types

More details: see [ThinkCS §9.2](#)

Collections are built up of smaller types:

- **lists**: ordered collection of 0 or more objects.
Mutable. Ex: `lst = [1, "love", "Python!"]`
- **tuples**: ordered collection of 0 or more objects.
Immutable. Ex: `my_tuple = (2, True, 4.96)`



Collection Types

More details: see [ThinkCS §9.2](#)

Collections are built up of smaller types:

- **lists**: ordered collection of 0 or more objects.
Mutable. Ex: `lst = [1, "love", "Python!"]`
- **tuples**: ordered collection of 0 or more objects.
Immutable. Ex: `my_tuple = (2, True, 4.96)`
- **strings**: ordered collection of 0 or more characters.
Immutable. Ex: `my_str = "1 love Python!"`



Collection Types

More details: see [ThinkCS §9.2](#)

Collections are built up of smaller types:

- **lists**: ordered collection of 0 or more objects. Mutable. Ex: `lst = [1, "love", "Python!"]`
- **tuples**: ordered collection of 0 or more objects. Immutable. Ex: `my_tuple = (2, True, 4.96)`
- **strings**: ordered collection of 0 or more characters. Immutable. Ex: `my_str = "1 love Python!"`
- **sets**: unordered collection of zero or more immutable objects. Mutable.



Collection Types

More details: see [ThinkCS §9.2](#)

Collections are built up of smaller types:

- **lists**: ordered collection of 0 or more objects. Mutable. Ex: `lst = [1, "love", "Python!"]`
- **tuples**: ordered collection of 0 or more objects. Immutable. Ex: `my_tuple = (2, True, 4.96)`
- **strings**: ordered collection of 0 or more characters. Immutable. Ex: `my_str = "1 love Python!"`
- **sets**: unordered collection of zero or more immutable objects. Mutable. Ex: `my_set = {3, 6, "cat", 4.5, False}`



Collection Types

More details: see [ThinkCS §9.2](#)

Collections are built up of smaller types:

- **lists**: ordered collection of 0 or more objects. Mutable. Ex: `lst = [1, "love", "Python!"]`
- **tuples**: ordered collection of 0 or more objects. Immutable. Ex: `my_tuple = (2, True, 4.96)`
- **strings**: ordered collection of 0 or more characters. Immutable. Ex: `my_str = "1 love Python!"`
- **sets**: unordered collection of zero or more immutable objects. Mutable. Ex: `my_set = {3, 6, "cat", 4.5, False}`
- **dictionaries**: collections of associated pairs of items where each pair is **key:value**. Mutable.



Collection Types

More details: see [ThinkCS §9.2](#)

Collections are built up of smaller types:

- **lists**: ordered collection of 0 or more objects. Mutable. Ex: `lst = [1, "love", "Python!"]`
- **tuples**: ordered collection of 0 or more objects. Immutable. Ex: `my_tuple = (2, True, 4.96)`
- **strings**: ordered collection of 0 or more characters. Immutable. Ex: `my_str = "1 love Python!"`
- **sets**: unordered collection of zero or more immutable objects. Mutable. Ex: `my_set = {3, 6, "cat", 4.5, False}`
- **dictionaries**: collections of associated pairs of items where each pair is **key:value**. Mutable. Ex: `my_dict = {"uno": 1, "dos": 2}`



This Chapter▼



One way to create a dictionary is to start with the empty dictionary and add **key-value pairs**. The empty dictionary is denoted `{}`.

Objects

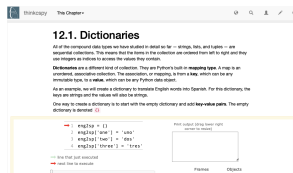
◀ ◻ ▶ ◀ ◻ ▶ ◀ ≡ ▶ ◀ ≡ ▶ ≡

Python: Dictionaries

- To create an empty dictionary:

```
counts = {}
```

```
translate = {'one': 'uno', 'two': 'dos'}
```



Python: Dictionaries

- To create an empty dictionary:

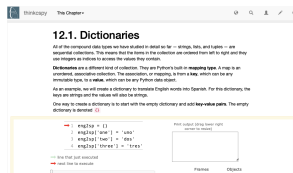
```
counts = {}
```

```
translate = {'one': 'uno', 'two': 'dos'}
```

- To add items to a dictionary:

```
translate["zero"] = "cero"
```

```
counts['Hunter'] = counts['Hunter'] + 1
```



Python: Dictionaries

- To create an empty dictionary:

```
counts = {}
```

```
translate = {'one': 'uno', 'two': 'dos'}
```

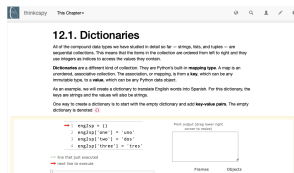
- To add items to a dictionary:

```
translate["zero"] = "cero"
```

```
counts['Hunter'] = counts['Hunter'] + 1
```

- Delete an item from the dictionary:

```
del translate['one']
```



Python: Dictionaries

- To create an empty dictionary:

```
counts = {}  
translate = {'one': 'uno', 'two': 'dos'}
```

- To add items to a dictionary:

```
translate["zero"] = "cero"  
counts['Hunter'] = counts['Hunter'] + 1
```

- Delete an item from the dictionary:

```
del translate['one']
```

- Get the length of a dictionary:

```
num_items = len(translate)
```



Python: Dictionaries

- To create an empty dictionary:

```
counts = {}  
translate = {'one': 'uno', 'two': 'dos'}
```

- To add items to a dictionary:

```
translate["zero"] = "cero"  
counts['Hunter'] = counts['Hunter'] + 1
```

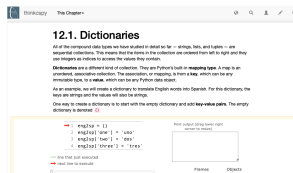
- Delete an item from the dictionary:

```
del translate['one']
```

- Get the length of a dictionary:

```
num_items = len(translate)
```

- Get the keys and values of a dictionary:



Python: Dictionaries

- To create an empty dictionary:

```
counts = {}  
translate = {'one': 'uno', 'two': 'dos'}
```

- To add items to a dictionary:

```
translate["zero"] = "cero"  
counts['Hunter'] = counts['Hunter'] + 1
```

- Delete an item from the dictionary:

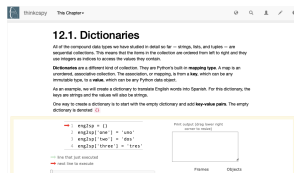
```
del translate['one']
```

- Get the length of a dictionary:

```
num_items = len(translate)
```

- Get the keys and values of a dictionary:

```
print(counts.keys(), count.values() )
```



Python: Dictionaries

- To create an empty dictionary:

```
counts = {}  
translate = {'one': 'uno', 'two': 'dos'}
```
- To add items to a dictionary:

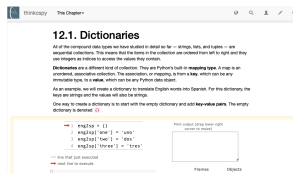
```
translate["zero"] = "cero"  
counts['Hunter'] = counts['Hunter'] + 1
```
- Delete an item from the dictionary:

```
del translate['one']
```
- Get the length of a dictionary:

```
num_items = len(translate)
```
- Get the keys and values of a dictionary:

```
print(counts.keys(), count.values() )
```
- Get both as pairs (key, value) of a dictionary:

```
counts.items()
```



Python: Dictionaries

- To create an empty dictionary:

```
counts = {}  
translate = {'one': 'uno', 'two': 'dos'}
```
- To add items to a dictionary:

```
translate["zero"] = "cero"  
counts['Hunter'] = counts['Hunter'] + 1
```
- Delete an item from the dictionary:

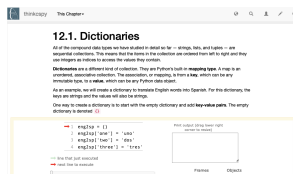
```
del translate['one']
```
- Get the length of a dictionary:

```
num_items = len(translate)
```
- Get the keys and values of a dictionary:

```
print(counts.keys(), count.values() )
```
- Get both as pairs (key, value) of a dictionary:

```
counts.items()
```
- Check if a key is in the dictionary:

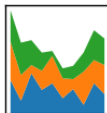
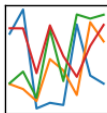
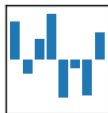
```
if 'uno' in translate: ...
```



Data Representation: Pandas

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$

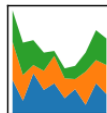
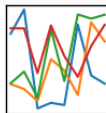
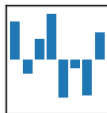


Pandas Documentation: [Getting Started](#)

Data Representation: Pandas

pandas

$$y_{it} = \beta' x_{it} + \mu_i + \epsilon_{it}$$



Pandas Documentation: [Getting Started](#)

- Constructing from a dictionary:

```
In [2]: df = pd.DataFrame(  
...:     {  
...:         "Name": [  
...:             "Braund, Mr. Owen Harris",  
...:             "Allen, Mr. William Henry",  
...:             "Bonnell, Miss. Elizabeth",  
...:         ],  
...:         "Age": [22, 35, 58],  
...:         "Sex": ["male", "male", "female"],  
...:     }  
...: )  
...:
```

In [3]: df

Out[3]:

	Name	Age	Sex
0	Braund, Mr. Owen Harris	22	male
1	Allen, Mr. William Henry	35	male
2	Bonnell, Miss. Elizabeth	58	female

Design Challenge

Are there two students with the same first name?

- ① Devise an algorithm to find duplicate names.
- ② Include your input and output for your algorithm.
- ③ Outline the process for finding duplicates.

Hint: Use a dictionary to store the names.

Recap

- Quick recap of a Python library, Plotly for creating interactive HTML maps.



Recap



- Quick recap of a Python library, Plotly for creating interactive HTML maps.
- More details on `while` loops for repeating commands for an indefinite number of times.

Recap



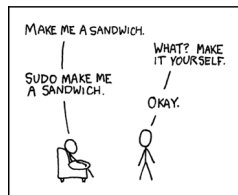
- Quick recap of a Python library, Plotly for creating interactive HTML maps.
- More details on `while` loops for repeating commands for an indefinite number of times.
- Introduced the max/min and linear-search design pattern.

Recap



- Quick recap of a Python library, Plotly for creating interactive HTML maps.
- More details on `while` loops for repeating commands for an indefinite number of times.
- Introduced the max/min and linear-search design pattern.
- Dictionaries: stores info as pairs: `key:value`.

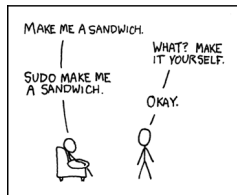
Final Exam Prep: UNIX



xkcd 149

- This course has three main themes:
 - ▶ Programming & Problem Solving
 - ▶ Organization of Hardware & Data
 - ▶ Design

Final Exam Prep: UNIX



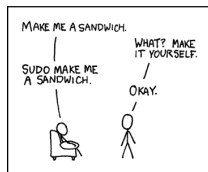
xkcd 149

- This course has three main themes:
 - ▶ Programming & Problem Solving
 - ▶ Organization of Hardware & Data
 - ▶ Design
- The operating system, Unix, is part of the second theme.
- Unix commands in the weekly on-line labs

Final Exam Prep: UNIX

Unix commands in the weekly on-line labs:

- *Lab 2:* `pwd`, `ls`, `mkdir`, `cd`
- *Lab 3:* `ls -l`, `cp`, `mv`
- *Lab 4:* `cd ../` (relative paths)
- *Lab 5:* `cd /usr/bin` (absolute paths), `cd ~`
- *Lab 6:* Scripts, `chmod`
- *Lab 7:* Running Python from the command line
- *Lab 8:* `git` from the command line
- *Lab 9:* `ls *.py` (wildcards)
- *Lab 10:* More on scripts, `vim`
- *Lab 11:* `ls | wc -c` (pipes), `grep`, `wc`



xkcd 149